

Sistemas Distribuídos e Computação em Nuvem

Aula 6 — IA como serviço distribuído (marco do curso)

C1 ·  Estudo Dirigido (EAD) · Lançamento do trabalho C1.A2

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Como funciona esta aula (EAD)

Esta é uma aula de **Estudo Dirigido (EAD)**: você percorre este roteiro **sozinho**, no seu ritmo, usando o **kit da C1** na sua máquina. Não há encontro presencial nesta semana.

- **O que fazer:** ler os conceitos, rodar o **roteiro guiado** com o kit e **anotar os tempos** medidos.
- **É o lançamento do C1.A2** 🚀 — ao fim, você terá o **repositório do trabalho criado com a parte REST funcionando**.
- **Dúvidas:** traga para a **Aula 7** (revisão + prova).

Onde estamos — a trilha do semestre

C1 · Fundamentos e comunicação (Aulas 1–7)

Sockets, concorrência, gRPC, REST → **IA como serviço**.

C2 · Coordenação e consistência (Aulas 8–12)

Mensageria, relógios lógicos, CAP, Raft, resiliência.

C3 · Nuvem, implantação e segurança (Aulas 13–18)

Containers, nuvem, serverless, observabilidade, segurança.

 **Aula 6** — o **marco**: tudo o que você construiu vira **um serviço de IA**.

Objetivos desta aula

Ao final, você será capaz de:

1. **Dominar** o vocabulário mínimo de IA para **operar** um modelo (sem matemática).
2. **Servir** um modelo de IA atrás de uma **API REST**.
3. **Reconhecer** os problemas distribuídos que a IA cria: **latência** e **cold start**.

Conceito 1/3 — O vocabulário mínimo de IA

Para **servir** um modelo você não precisa de matemática — precisa de **cinco palavras**:

- **Modelo**: um **arquivo já treinado** que transforma uma entrada em uma saída.
- **Inferência**: o ato de **usar** o modelo para obter uma resposta — na prática, **chamar uma função**.
- **Embedding**: um texto como **lista de números** que captura o significado (permite comparar por semelhança).
- **Token**: um **pedaço de texto** que o modelo processa por vez (uma palavra ou parte dela).
- **LLM**: *Large Language Model* — modelo treinado em enormes volumes de texto, **consumido por API**.

💡 Com essas cinco palavras você opera qualquer serviço de IA **sem abrir a caixa-preta**. A sua nota vem da **engenharia**, não do modelo.

Conceito 2/3 — Servir um modelo como serviço

- **Regra de ouro:** carregar o modelo **UMA única vez**, quando o serviço sobe, e **mantê-lo em memória**.
- Carregá-lo **a cada requisição** seria um **erro grave** — é uma operação **cara** (ler um arquivo grande do disco e prepará-lo).
- Feito o carregamento na inicialização, a rota `/predict` apenas **recebe a entrada e chama a inferência**.

💡 A API é a **mesma** da Aula 5 (REST/FastAPI). O que muda é a **carga de trabalho** que ela executa por baixo. No kit, isso já está pronto no `startup` do `app/api_rest.py`.

Conceito 3/3 — Os problemas que a IA traz

- **Latência:** a inferência é **lenta** comparada a uma consulta comum — pense em **centenas de ms ou segundos**, não microssegundos.
- **Cold start:** a **primeira chamada** depois de o serviço subir é sempre a **mais lenta** (o ambiente ainda está sendo preparado: modelo carregando, memória alocando).
- Como a operação é lenta, **deixar o cliente parado esperando** é um mau desenho.

⚠ A solução — processar de forma **assíncrona com uma fila** — é a **Aula 8**. É por ser uma carga **pesada, lenta e realista** que a IA é um caso de estudo tão bom: ela expõe, na prática, todos os problemas da teoria.

Roteiro guiado (EAD)

Seu modelo atrás de uma API — usando o kit da C1

Roteiro · Passo 1 — suba o kit da C1

Se já clonou na Aula 1, é só entrar na pasta e ativar o ambiente. Se não:

```
git clone https://github.com/howardroatti/sd-2026-2-kit-c1a2.git
cd sd-2026-2-kit-c1a2
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.api_rest:app --reload --port 8000    # abra http://localhost:8000/docs
```

💡 Repare no log de subida: [startup] modelo carregado em X.XXXs . Esse é o **modelo sendo carregado UMA vez** (a regra de ouro). Anote esse tempo — é o seu **cold start**.

Roteiro · Passo 2 — chame a inferência

No `/docs` (ou por outro terminal com o `.venv` ativo):

```
python exemplos/cliente_rest.py "o atendimento foi otimo"  
# sincrono: {'texto': '...', 'sentimento': 'positivo', 'confianca': 0.57, 'tempo_ms': 0.5}
```

- A rota `/predict-sync` recebe o texto e **chama a inferência** do modelo (`app/modelo.py`).
- O modelo é um **classificador de sentimento** (positivo/negativo) — a **tarefa** que o seu sistema distribuído executa.

💡 O `tempo_ms` na resposta é o tempo **só da inferência**. Compare com o tempo **total** (ida-e-volta) que você mede no próximo passo.

Roteiro · Passo 3 — meça o tempo das requisições

O modelo já foi **carregado no startup** (veja o log `[startup] modelo carregado em ...s` — **esse** é o custo de carga). Agora rode o medidor (`exemplos/aula06/medir_tempos.py`), que chama `/predict-sync` **10 vezes**:

```
python medir_tempos.py
# chamada 1: 8.1 ms -> positivo
# chamada 2: 2.5 ms -> positivo
# ...
# 1a chamada: 8.1 ms
# media das seguintes: 9.1 ms
```

💡 Os tempos são **baixos e estáveis** — é a **regra de ouro funcionando**: carregou uma vez no startup, então cada requisição é rápida.

⚠️ Com um modelo **pesado** (um LLM), a 1ª chamada poderia levar **segundos** (cold start) e cada inferência seria lenta — daí a fila/assíncrono da **Aula 8**. **Anote** os seus tempos para o relatório.

O mapa do kit — onde cada aula entra

O kit `sd-2026-2-kit-c1a2` **junta tudo** o que você viu em C1:

Peça do kit	O que é	Aula
<code>app/api_rest.py</code>	interface REST (<code>/predict</code> , <code>/resultado/{id}</code>)	5
<code>app/servidor_grpc.py</code> + <code>proto/</code>	interface gRPC (<code>Prever</code> , <code>PreverLote</code>)	4
<code>app/worker.py</code>	processa em segundo plano (threads/fila)	3 / 8
<code>app/fila.py</code>	a fila (assíncrono)	8
<code>app/modelo.py</code>	o modelo (pronto — não mexa)	6

💡 Você não vai **reescrever** o kit: vai **completar as TAREFAS** (veja `TAREFAS.md`), cada uma apontando o arquivo e a aula.

Lançamento do C1.A2 — Serviço de Inferência Distribuído

Aqui o trabalho **ganha vida**: o modelo (pronto no kit) passa a ser **servido de verdade**, e as **duas interfaces** devem **convergir para o mesmo resultado**.

- **Carregar o modelo UMA vez**, na subida do serviço (já feito no kit).
- **Conferir** que **REST e gRPC** devolvem o **mesmo resultado** para a **mesma entrada**.
- **TAREFA 7** — escrever o **README** explicando **sua** arquitetura e como executar do zero.

 Rubrica (lembrete): arquitetura 1,5 · comunicação 1,5 · resiliência 1,0 · execução reproduzível 1,0. A **sofisticação do modelo NÃO pontua**.

Entregável desta aula (EAD)

1. **Crie o repositório** do seu C1.A2 (pode partir do kit) e faça o **primeiro commit**.
2. Deixe a **parte REST funcionando**: `uvicorn app.api_rest:app` sobe e o `/predict-sync` responde.
3. **Anote os tempos** (cold start × warm) do Passo 3 — vão para o relatório.
4. Leia o `TAREFAS.md` e marque por onde vai começar (dica: **TAREFA 2**, `GET /resultado/{id}`, usa o que você viu na Aula 5).

📌 **Meta do EAD:** repositório do **C1.A2 criado**, com a **parte REST já funcionando**. Traga dúvidas para a Aula 7.

◆ Foco ENADE

Aqui o ENADE cobra **menos a IA em si e mais a arquitetura**:

- **Aplicações e casos de uso** de sistemas distribuídos.
- **Arquitetura de serviços e separação de responsabilidades**.
- **Desempenho, latência e tempo de resposta** em serviços.
- **Tecnologias emergentes: IA como serviço**.

Termos-chave: Inferência · Modelo · Embedding · Token · LLM · Cold start

💡 Domine **modelo, inferência, embedding, token, LLM** e **cold start** no sentido **operacional** — não na matemática.

Questão de autoavaliação (estilo ENADE)

Um serviço de inferência está **lento**; verifica-se que o **modelo é carregado do disco a cada requisição**. A correção mais adequada é:

- A) Substituir REST por **SOAP** na interface.
- B) **Carregar o modelo uma única vez**, na inicialização, mantendo-o em **memória**.
- C) **Reduzir** o número de rotas expostas.
- D) Trocar o transporte de **TCP para UDP**.
- E) **Remover** a validação dos dados de entrada.

Resolução — alternativa B

- Carregar o modelo é uma operação **cara** e deve ocorrer **uma única vez**, no **startup**.
- As demais **não atacam a causa** do problema (protocolo, número de rotas, transporte, validação).

💡 É a **regra de ouro** do Conceito 2 — e o que o kit já faz no `startup` do `api_rest.py`.

Fora da sala · Glossário

Para estudar

- **Tanenbaum & Van Steen**, cap. 1 — casos de aplicação.
- `README.md` e `TAREFAS.md` do **kit da C1**.
- Compare `/predict-sync` (síncrono, hoje) com o que virá **assíncrono** na Aula 8.

Glossário

- **Modelo**: arquivo treinado (entrada → saída).
- **Inferência**: usar o modelo (chamar uma função).
- **Embedding**: texto como vetor de números.
- **Token**: pedaço de texto processado por vez.
- **LLM**: modelo de linguagem grande, via API.
- **Cold start**: lentidão da 1ª execução.

Bom estudo dirigido! 🚀

Traga o repositório do C1.A2 (REST funcionando) e suas dúvidas para a Aula 7.

Próxima (Aula 7): Revisão e Avaliação C1.A1 (prova estilo ENADE) + entrega do C1.A2.

 [Voltar ao índice](#)
todas as aulas